

Full Documentation Bundle

Generated from the current README, handbook, tooling notes, and tutorial set.

Included Documents

- README.md
- docs/handbook.md
- docs/tooling_decisions.md
- docs/tutorials/00_learning_path.md
- docs/tutorials/01_problem_and_embedding_foundations.md
- docs/tutorials/02_data_preparation_and_schema.md
- docs/tutorials/03_pair_mining_and_splits.md
- docs/tutorials/04_contrastive_training_and_lora.md
- docs/tutorials/05_backend_selection_peft_vs_unsloth.md
- docs/tutorials/06_retrieval_systems_bm25_dense_hybrid.md
- docs/tutorials/07_evaluation_metrics_and_judging.md
- docs/tutorials/08_embedding_space_visualization.md
- docs/tutorials/09_rag_chroma_and_pinecone.md
- docs/tutorials/10_graphrag_local_and_global.md
- docs/tutorials/11_inference_and_serving_patterns.md
- docs/tutorials/12_reports_validation_and_auditability.md

README

Domain-Adapted Embedding Alignment

Production-oriented local pipeline for adapting embedding behavior across medical, legal, and cybersecurity retrieval workloads.

This repository prepares real multi-domain corpora, trains a compact embedding model with adapter-based fine-tuning, benchmarks sparse/dense/hybrid retrieval plus RAG/GraphRAG behavior, and emits auditable completion artifacts.

What This Project Does

- Builds real documents, queries, and contrastive pairs artifacts from public datasets.
- Trains Qwen/Qwen3-Embedding-0.6B with LoRA using backend routing (peft or unsloth).

- Evaluates BM25, dense baseline, dense tuned, and hybrid retrieval with latency + quality metrics.
- Benchmarks Chroma and optional Pinecone RAG paths.
- Benchmarks GraphRAG local/global retrieval behavior.
- Generates machine-readable final reports and completion checks.

Verified Run Snapshot (Ground Truth)

Source of truth:

- `artifacts/reports/data_preparation_report.json`
- `artifacts/reports/training_report.json`
- `artifacts/evaluation/retrieval_evaluation.json`
- `artifacts/evaluation/rag_benchmark.json`
- `artifacts/evaluation/graphrag_benchmark.json`
- `artifacts/reports/completion_checklist.json`

Data Preparation

- Documents: **168,448**
- Queries: **98,907**
- Pairs: **200,000**
- Domain document mix: general 153,637 | medical 4,392 | legal 7,999 | cybersecurity 2,420

Training

- Backend requested/used: **unsloth / unsloth**
- Total steps: **700**
- Best validation loss: **0.0694618**
- Train runtime: **5120.97s**
- Eval runtime: **231.18s**

Retrieval (selected)

- `bm25` : recall@10 **0.9667**, MRR **0.7691**, mean latency **0.86ms**
- `baseline_dense_qwen4b` : recall@10 **1.0000**, MRR **0.8542**, mean latency **2902.25ms**
- `tuned_dense_qwen0_6b_lora` : recall@10 **0.9750**, MRR **0.7556**, mean latency **47.33ms**
- `hybrid_tuned` : recall@10 **0.9833**, MRR **0.8107**, mean latency **56.04ms**

Similarity diagnostics (`retrieval_evaluation.json`):

- Ranking accuracy: baseline **0.8933** -> tuned **0.9700**

RAG / GraphRAG

- Chroma hit@5: baseline **1.0000**, tuned **0.9417**
- Pinecone: **disabled** in this run (`"executed": false`)

- GraphRAG local/global hit@5:
 - baseline **0.9467 / 0.4600**
 - tuned **0.8800 / 0.4600**

Completion

- `project_completion_ok`: **true**

Architecture Map

Core package: `src/domain_adapted_embedding_alignment/`

- `data/` : dataset loaders and pair mining
- `training/` : PEFT trainer, Unsloth trainer, backend resolver
- `retrieval/` : BM25, dense retriever, hybrid fusion, embedding backends
- `evaluation/` : metrics, judge integration, embedding-space diagnostics
- `rag/` : Chroma/Pinecone benchmark helpers and inference formatting
- `graphrag/` : graph construction and local/global retrieval
- `pipelines/` : stage orchestration and final report assembly
- CLI entrypoint: `cli.py`

Setup (uv only)

```
cd /home/ahmad/AI/Domain-Adapted-Embedding-Alignment
uv python install 3.12.10
uv venv --python 3.12.10 .venv
source .venv/bin/activate
uv sync
```

Optional extras:

```
uv sync --extra unsloth
uv sync --extra trl
```

Runtime Requirements

Start Ollama:

```
ollama serve
```

Pull models used by the workflows:

```
ollama pull qwen3-embedding:4b
ollama pull qwen3.5:4b
ollama pull granite4.1:3b
ollama pull glm-ocr
```

Optional Pinecone requires `PINECONE_API_KEY`.

Run the Pipeline

Full pipeline:

```
dea-cli run-all
```

Stage-by-stage:

```
dea-cli prepare-data
dea-cli train --training-backend auto
dea-cli evaluate
dea-cli rag-benchmark
dea-cli graphrag-benchmark
dea-cli inference --query "What treatments are available for a brain tumor?"
dea-cli build-final-report
dea-cli finalize-reports
```

Notebook execution:

```
bash scripts/execute_notebooks.sh
```

Key Artifacts

- Data and training:
 - `data/processed/documents.parquet`
 - `data/processed/queries.parquet`
 - `data/processed/pairs.parquet`
 - `artifacts/reports/training_report.json`
- Evaluation and benchmarks:
 - `artifacts/evaluation/retrieval_evaluation.json`
 - `artifacts/evaluation/benchmark_table.csv`
 - `artifacts/evaluation/rag_benchmark.json`
 - `artifacts/evaluation/graphrag_benchmark.json`
- Finalization:
 - `artifacts/reports/final_report.json`

- `artifacts/reports/completion_checklist.json`
- `artifacts/reports/tool_impact_report.json`

Documentation

- Handbook: `docs/handbook.md`
- Tutorials: `docs/tutorials/00_learning_path.md` through `docs/tutorials/12_reports_validation_and_auditability.md`
- Tooling rationale: `docs/tooling_decisions.md`
- Unified bundle: `docs/documentation.md`
- PDF: `docs/documentation.pdf`

Operational Notes

- Judge model parsing can fail intermittently; fallback judge scores are neutral by design (`evaluation/llm_judge.py`).
- Pinecone is optional and run-dependent.
- Latency numbers are environment/profile dependent (`settings.py`, `configs/local_6gb.yaml`).

Handbook

Domain-Adapted Embedding Alignment Handbook

Implementation-level handbook for the current repository state and completed run artifacts.

Table of Contents

1. [Project Goal and Scope](#)
2. [End-to-End Workflow](#)
3. [Configuration and Runtime Profiles](#)
4. [Data Preparation](#)
5. [Pair Mining and Split Strategy](#)
6. [Training Backends: PEFT and Unsloth](#)
7. [Retrieval Stack: BM25, Dense, Hybrid](#)
8. [Evaluation, Judging, and Diagnostics](#)
9. [RAG Benchmarking \(Chroma + Pinecone\)](#)
10. [GraphRAG Benchmarking](#)
11. [Inference Path](#)
12. [Final Reporting and Completion Gates](#)

13. [Testing and Reliability](#)
14. [Operational Caveats](#)
15. [References](#)

Project Goal and Scope

What it is

A local, reproducible pipeline to adapt embedding behavior for domain-sensitive retrieval tasks (medical, legal, cybersecurity) while preserving operational auditability.

Why it is used

Generic embeddings often underperform when domain vocabulary shifts from common language. This pipeline aligns embedding space around domain relevance signals and reports quality/latency tradeoffs end-to-end.

How it appears in code

- CLI command router: `src/domain_adapted_embedding_alignment/cli.py`
- Full orchestration:
`src/domain_adapted_embedding_alignment/pipelines/run_end_to_end.py`

Practical implementation view

Each stage emits persisted artifacts consumed by downstream stages and by completion checks. This design makes runs inspectable and restartable at stage boundaries.

End-to-End Workflow

What it is

Stage sequence used by `dea-cli run-all`:

1. `prepare-data`
2. `train`
3. `evaluate`
4. `rag-benchmark`
5. `graphrag-benchmark`
6. `inference`
7. `build-final-report`
8. `finalize-reports`

Why it is used

Separate stages reduce blast radius, support partial reruns, and keep evidence traceable in artifacts/.

How it appears in code

- Stage orchestration:
`src/domain_adapted_embedding_alignment/pipelines/run_end_to_end.py`
- Stage implementations:
`src/domain_adapted_embedding_alignment/pipelines/*.py`

Practical implementation view

The same stage implementations can be called independently from CLI for targeted debugging and benchmarking.

Configuration and Runtime Profiles

What it is

Typed runtime config via Pydantic settings with `DEA_` environment overrides.

Why it is used

Keeps behavior reproducible and hardware-aware without editing source files.

How it appears in code

- Config model: `src/domain_adapted_embedding_alignment/settings.py`
- Profiles: `configs/local_6gb.yaml`, `configs/smoke_cpu.yaml`

Practical implementation view

Key run values in the completed run:

- `final_pair_target=200000`
- `train_max_steps=700`
- `eval_query_limit=120`
- `rag_query_limit=120`
- `graphrag_query_limit=150`

Data Preparation

What it is

Loads real corpora and standardizes them into canonical parquet artifacts.

Why it is used

Reliable training/evaluation requires unified schema and deterministic preprocessing across heterogeneous sources.

How it appears in code

- Pipeline entry: `src/domain_adapted_embedding_alignment/pipelines/prepare_data.py`
- Loaders: `src/domain_adapted_embedding_alignment/data/loaders.py`
- Typed records: `src/domain_adapted_embedding_alignment/schemas.py`

Practical implementation view (run outputs)

From `artifacts/reports/data_preparation_report.json`:

- `documents_total`: 168448
- `queries_total`: 98907
- `pairs_total`: 200000
- `documents_by_domain`: general 153637, medical 4392, legal 7999, cybersecurity 2420

Pair Mining and Split Strategy

What it is

Contrastive pair construction with positive, random negative, and hard negative text per query.

Why it is used

Hard negatives are required to learn meaningful boundaries in embedding space; deterministic split assignment prevents accidental leakage.

How it appears in code

- Builder: `src/domain_adapted_embedding_alignment/data/pair_builder.py`
- Split function: `_split_from_query_id`
- Hard negative function: `_pick_hard_negative`
- Domain quotas: `_domain_quota`

Practical implementation view

Current implementation uses lexical-overlap sampled hard negatives with caching, avoiding full-corpus per-pair BM25 rescoring at 200k-pair scale.

Training Backends: PEFT and Unsloth

What it is

Backend routing chooses either:

- PEFT path (`ContrastiveTrainer`)
- Unsloth path (`UnslothContrastiveTrainer` with `SentenceTransformers` trainer)

Why it is used

Allows stable fallback behavior while exploiting acceleration when environment supports Unsloth.

How it appears in code

- Resolver:
`src/domain_adapted_embedding_alignment/training/backend_selection.py`
- Training entry:
`src/domain_adapted_embedding_alignment/pipelines/train_model.py`
- Unsloth trainer:
`src/domain_adapted_embedding_alignment/training/unsloth_trainer.py`
- PEFT trainer: `src/domain_adapted_embedding_alignment/training/trainer.py`

Practical implementation view (run outputs)

From `artifacts/reports/training_report.json`:

- `runtime.backend_requested`: `unsloth`
- `runtime.backend_used`: `unsloth`
- `runtime.fallback_reason`: `null`
- `total_steps`: 700
- `best_val_loss`: 0.06946182250976562
- `train_runtime_seconds`: 5120.9695

Retrieval Stack: BM25, Dense, Hybrid

What it is

Five retrieval systems are evaluated:

- BM25 sparse retrieval
- Dense baseline (`qwen3-embedding:4b`)
- Dense tuned (`Qwen3-Embedding-0.6B` + adapter)
- Hybrid baseline
- Hybrid tuned

Why it is used

Retrieval quality and latency profiles vary sharply between sparse and dense methods; hybrid fusion can stabilize performance.

How it appears in code

- Backend construction:
`src/domain_adapted_embedding_alignment/retrieval/backend_factory.py`
- Embedding backends:
`src/domain_adapted_embedding_alignment/retrieval/embeddings.py`
- BM25: `src/domain_adapted_embedding_alignment/retrieval/bm25.py`
- Dense retrieval: `src/domain_adapted_embedding_alignment/retrieval/dense.py`
- Fusion: `src/domain_adapted_embedding_alignment/retrieval/hybrid.py`

Practical implementation view (run outputs)

From `artifacts/evaluation/retrieval_evaluation.json`:

- BM25 recall@10 0.9667, mean latency 0.86ms
- Baseline dense recall@10 1.0000, mean latency 2902.25ms
- Tuned dense recall@10 0.9750, mean latency 47.33ms
- Hybrid tuned recall@10 0.9833, mean latency 56.04ms

Interpretation: this run shows a strong quality-latency tradeoff rather than a single global winner.

Evaluation, Judging, and Diagnostics

What it is

Evaluation layer combining retrieval metrics, latency summaries, RAG proxies, optional LLM judge scoring, and embedding-space diagnostics.

Why it is used

Production retrieval decisions require both quality and operational evidence.

How it appears in code

- Evaluation pipeline:
`src/domain_adapted_embedding_alignment/pipelines/evaluate_models.py`
- Core evaluator:
`src/domain_adapted_embedding_alignment/evaluation/evaluator.py`
- Judge fallback logic:
`src/domain_adapted_embedding_alignment/evaluation/llm_judge.py`
- Metric helpers:
`src/domain_adapted_embedding_alignment/retrieval/metrics.py`
- Embedding projections:
`src/domain_adapted_embedding_alignment/evaluation/visualization.py`

Practical implementation view (run outputs)

From `artifacts/evaluation/retrieval_evaluation.json`:

- Similarity ranking accuracy improved from 0.8933 to 0.9700 (baseline -> tuned)
- Judge metrics are present for baseline and tuned dense systems

From `artifacts/logs/02_evaluate.log`:

- Multiple judge parse failures were logged for `qwen3.5:4b`
- Fallback scores remained neutral by design, keeping evaluation execution stable

RAG Benchmarking (Chroma + Pinecone)

What it is

RAG benchmark layer over local Chroma collections plus optional Pinecone path.

Why it is used

RAG retrieval behavior can differ from standalone retrieval benchmarking due to indexing/runtime interactions.

How it appears in code

- Pipeline:
`src/domain_adapted_embedding_alignment/pipelines/run_rag_benchmarks.py`
- Chroma helpers: `src/domain_adapted_embedding_alignment/rag/chroma_demo.py`
- Pinecone helpers:
`src/domain_adapted_embedding_alignment/rag/pinecone_demo.py`

Practical implementation view (run outputs)

From `artifacts/evaluation/rag_benchmark.json`:

- Chroma baseline hit@5: 1.0
- Chroma tuned hit@5: 0.9416666666666667
- Chroma mean latency: 159.95ms baseline vs 45.06ms tuned
- Pinecone: { "executed": false, "reason": "disabled" }

GraphRAG Benchmarking

What it is

Graph-enhanced retrieval using a document-term graph with local neighborhood expansion and global community retrieval.

Why it is used

Graph traversal provides structural context beyond plain nearest-neighbor lookup.

How it appears in code

- Graph construction:
`src/domain_adapted_embedding_alignment/graphrag/graph_builder.py`
- Local/global retrieval:
`src/domain_adapted_embedding_alignment/graphrag/retriever.py`
- Benchmark pipeline:
`src/domain_adapted_embedding_alignment/pipelines/run_graphrag_benchmarks.py`

Practical implementation view (run outputs)

From `artifacts/logs/04_graphrag.log`:

- Graph built with 9220 nodes, 187729 edges, 6 communities

From `artifacts/evaluation/graphrag_benchmark.json`:

- Baseline local/global hit@5: 0.9467 / 0.46
- Tuned local/global hit@5: 0.88 / 0.46
- Query count: 150

Inference Path

What it is

Query-time retrieval endpoint returning ranked results with metadata and previews.

Why it is used

Represents end-user retrieval behavior after training and indexing stages.

How it appears in code

- Inference stage:
`src/domain_adapted_embedding_alignment/pipelines/run_inference.py`
- Output formatting: `src/domain_adapted_embedding_alignment/rag/inference.py`

Practical implementation view (run outputs)

From `artifacts/logs/05_inference.log`:

- Example queries executed: 3
- Mean latency: 65.10ms
- Backend used: `HuggingFaceEmbeddingBackend`
- Outputs include `rank`, `doc_id`, `score`, `domain`, `source`, `preview`

Final Reporting and Completion Gates

What it is

Final aggregation and validation layer that determines run completeness.

Why it is used

Prevents ambiguous "done" status by checking artifacts and runtime metadata explicitly.

How it appears in code

- Final report assembly:
`src/domain_adapted_embedding_alignment/pipelines/build_final_report.py`
- Completion/tool impact reports:
`src/domain_adapted_embedding_alignment/pipelines/finalize_reports.py`

Practical implementation view (run outputs)

From `artifacts/reports/completion_checklist.json`:

- `all_required_artifacts_ok: true`
- `all_runtime_checks_ok: true`
- `project_completion_ok: true`

From `artifacts/reports/tool_impact_report.json`:

- `unsloth.used: true`
- `peft.used: false`
- `trl.used_in_runtime: false`

Testing and Reliability

What it is

Unit-level checks for retrieval metrics, backend selection, pair splits, report finalization, and notebook scaffolding.

Why it is used

Protects correctness in core evaluation math and completion criteria.

How it appears in code

- Test suite: `tests/`

Practical implementation view (run outputs)

From `artifacts/logs/09_pytest.log`:

- 12 tests passed.

Operational Caveats

- Latency values are hardware- and profile-dependent.
- LLM judge failures do not crash evaluation; fallback scores are neutral.
- Pinecone path is optional and controlled by runtime settings.
- Baseline dense and tuned dense may trade quality for latency differently by domain and query distribution.

References

Project-local references

- README.md
- docs/tooling_decisions.md
- artifacts/reports/final_report.json
- artifacts/reports/completion_checklist.json
- artifacts/logs/

External references

- Unsloth embedding fine-tuning: <https://unsloth.ai/docs/basics/embedding-finetuning>
- PEFT task types (FEATURE_EXTRACTION): https://huggingface.co/docs/peft/main/package_reference/peft_types
- TRL overview: <https://huggingface.co/docs/trl>
- Sentence Transformers losses: https://www.sbert.net/docs/package_reference/sentence_transformer/losses.html
- Ollama embed API: <https://docs.ollama.com/api/embed>
- Chroma docs: <https://docs.trychroma.com/>
- Pinecone docs: <https://docs.pinecone.io/>
- Microsoft GraphRAG docs: <https://github.com/microsoft/graphrag/blob/main/docs/query/overview.md>

Tooling Decisions

Tooling Decisions: PEFT, Unsloth, and TRL

This note explains why the runtime uses PEFT/Unsloth for embedding alignment and why TRL is documentation-only for this project version.

1) PEFT

Definition

PEFT (Parameter-Efficient Fine-Tuning) updates a small subset of parameters (for example LoRA adapters) instead of full model weights.

Why it is used here

- Embedding alignment must run under local compute constraints.
- Adapter-based tuning keeps memory and storage overhead practical.
- It enables direct baseline-vs-tuned comparison on the same base model family.

Where it appears in code

- LoRA-centric training path:
`src/domain_adapted_embedding_alignment/training/trainer.py`
- Adapter-aware loading for retrieval:
`src/domain_adapted_embedding_alignment/retrieval/backend_factory.py`

Practical effect to measure

- Retrieval deltas (`recall@k` , `MRR` , `NDCG` , `MAP`)
- Similarity ranking improvement
- Latency vs quality tradeoff relative to baseline

2) Unsloth

Definition

Unsloth is an acceleration framework for efficient fine-tuning, with embedding-specific guidance centered on SentenceTransformer workflows.

Why it is used here

- Provides an accelerated optional backend for embedding training.
- Enables strict backend routing with explicit fallback semantics.
- Preserves the same objective while improving practical runtime throughput on compatible GPUs.

Where it appears in code

- Backend resolution rules:
`src/domain_adapted_embedding_alignment/training/backend_selection.py`
- Unsloth trainer implementation:
`src/domain_adapted_embedding_alignment/training/unsloth_trainer.py`
- Runtime metadata persistence:
`src/domain_adapted_embedding_alignment/pipelines/train_model.py`

Practical effect to measure

- Train runtime and throughput against PEFT path
- Stability of downstream retrieval quality and latency
- Backend request/use/fallback traceability in `training_report.json`

3) TRL

Definition

TRL provides post-training trainers for language model alignment workflows (for example SFT, DPO, PPO/GRPO family).

Why it is not in runtime for this project

- This repository targets encoder-style contrastive embedding alignment.
- Runtime objective is not causal language-model post-training.
- Adding TRL trainers here would increase complexity without solving the current objective.

Where it is covered

- Documentation and optional dependency path for future scope expansion.
- Tool impact report records TRL as `documentation_only`.

Practical effect to measure

- No direct runtime metric impact in this version by design.
- Reduced objective mismatch and maintenance surface.

Official Sources

- Unsloth embedding fine-tuning docs: <https://unsloth.ai/docs/basics/embedding-finetuning>
- Unsloth requirements: <https://unsloth.ai/docs/get-started/fine-tuning-for-beginners/unsloth-requirements>
- PEFT type references (`FEATURE_EXTRACTION`):
https://huggingface.co/docs/peft/main/package_reference/peft_types
- PEFT overview: <https://huggingface.co/docs/peft/index>
- TRL overview: <https://huggingface.co/docs/trl>
- TRL SFT trainer docs: https://huggingface.co/docs/trl/en/sft_trainer
- TRL DPO trainer docs: https://huggingface.co/docs/trl/en/dpo_trainer
- Sentence Transformers losses:
https://www.sbert.net/docs/package_reference/sentence_transformer/losses.html

Tutorial: 00_learning_path

00 - Learning Path

What it is

A structured route through the repository from first principles to production-style operation.

Why it is used

This project spans data engineering, contrastive training, retrieval science, benchmarking, and audit reporting. A defined sequence prevents fragmented understanding.

How it appears in code

- Workflow entrypoint: `src/domain_adapted_embedding_alignment/cli.py`
- Stage orchestration:
`src/domain_adapted_embedding_alignment/pipelines/run_end_to_end.py`
- Executable notebook track:
 - `notebooks/01_problem_analysis_and_embedding_theory.ipynb`
 - `notebooks/02_data_preparation_and_pair_mining.ipynb`
 - `notebooks/03_baseline_retrieval_systems.ipynb`
 - `notebooks/04_contrastive_learning_and_finetuning.ipynb`
 - `notebooks/05_evaluation_and_error_analysis.ipynb`
 - `notebooks/06_visualization_embedding_space.ipynb`
 - `notebooks/07_rag_integration_chroma_pinecone.ipynb`
 - `notebooks/08_graphrag_integration_and_inference.ipynb`

Practical path

1. Read `README.md` for the system picture and run snapshot.
2. Read `docs/handbook.md` for implementation depth.
3. Complete tutorials 01 to 05 to master data + training internals.
4. Complete tutorials 06 to 10 to master retrieval/RAG/GraphRAG behavior.
5. Complete tutorials 11 and 12 to understand inference and completion gates.

Real run anchor points

Use these artifacts as your ground truth while studying:

- Data: 168448 docs, 98907 queries, 200000 pairs
- Training: 700 steps, backend used `unsloth`
- Completion: `project_completion_ok=true`

Files:

- `artifacts/reports/data_preparation_report.json`
- `artifacts/reports/training_report.json`
- `artifacts/reports/completion_checklist.json`

Next tutorial

Proceed to `docs/tutorials/01_problem_and_embedding_foundations.md`.

Tutorial:

01_problem_and_embedding_foundations

01 - Problem and Embedding Foundations

What it is

Domain-adapted embedding alignment means training an embedding model so semantically related items in your specific domain are closer in vector space than irrelevant but lexically similar items.

Why it is used

General-purpose embeddings can miss domain nuance. Terms that look close in generic corpora are not always close for downstream retrieval objectives in medicine, law, or cybersecurity.

How it appears in code

- Model and runtime defaults:
`src/domain_adapted_embedding_alignment/settings.py`
- Embedding backend implementations:
`src/domain_adapted_embedding_alignment/retrieval/embeddings.py`
- Baseline/tuned backend routing:
`src/domain_adapted_embedding_alignment/retrieval/backend_factory.py`
- Training objective entrypoints:
 - `src/domain_adapted_embedding_alignment/training/trainer.py`
 - `src/domain_adapted_embedding_alignment/training/unsloth_trainer.py`

Practical explanation from run outputs

From `artifacts/evaluation/retrieval_evaluation.json`:

- Baseline dense quality is stronger on top-level recall/MRR in this run.

- Tuned dense is dramatically faster (47.33ms mean vs 2902.25ms mean).
- Similarity ranking accuracy improved (0.8933 -> 0.9700) for sampled pair diagnostics.

This is the core lesson: adaptation is a systems tradeoff problem, not a single-metric contest.

Beginner checkpoint

- Understand cosine similarity at a conceptual level.
- Understand why positive and hard-negative pairs are needed.
- Understand why you must report both quality and latency.

Advanced checkpoint

- Explain why retrieval metrics and pairwise ranking diagnostics can disagree.
- Explain why backend choice (ollama baseline vs tuned HF adapter) can dominate latency behavior.

Tutorial: 02_data_preparation_and_schema

02 - Data Preparation and Schema

What it is

Data preparation loads multiple real datasets and normalizes them into a shared schema:

- documents.parquet
- queries.parquet
- pairs.parquet

Why it is used

Training and evaluation stages depend on consistent fields and deterministic generation. Without schema stability, downstream metrics and comparisons become unreliable.

How it appears in code

- Pipeline stage:
src/domain_adapted_embedding_alignment/pipelines/prepare_data.py
- Dataset-specific loaders:
src/domain_adapted_embedding_alignment/data/loaders.py
 - load msmarco
 - load medical_medmentions

- `load_legal_lexglue`
- `load_cyber_mitre`
- Record contracts: `src/domain_adapted_embedding_alignment/schemas.py`

Practical explanation from run outputs

From `artifacts/reports/data_preparation_report.json`:

- `documents_total`: 168448
- `queries_total`: 98907
- `pairs_total`: 200000
- document mix:
 - `general`: 153637
 - `medical`: 4392
 - `legal`: 7999
 - `cybersecurity`: 2420

These values are the canonical evidence for this run's training and benchmark scope.

Beginner checkpoint

- Learn which file stores documents vs queries vs pairs.
- Learn where domain labels live and why they matter for analysis.

Advanced checkpoint

- Explain how changing `DEA_MS MARCO_MAX_ROWS` or `DEA_FINAL_PAIR_TARGET` changes model behavior and evaluation confidence.
- Identify where data lineage is persisted (`data_preparation_report.json`).

Common pitfalls

- Missing MITRE CTI source breaks cyber ingestion.
- Inconsistent environment overrides can silently shift dataset composition.

Tutorial: 03_pair_mining_and_splits

03 - Pair Mining and Splits

What it is

Pair mining transforms query relevance seeds into contrastive training rows containing:

- `query`
- `positive_text`
- `negative_text`
- `hard_negative_text`
- `deterministic_split`

Why it is used

Contrastive learning quality depends on how informative negatives are. Easy negatives teach little; hard negatives force useful representation boundaries.

How it appears in code

- Pair builder: `src/domain_adapted_embedding_alignment/data/pair_builder.py`
- Deterministic split assignment: `_split_from_query_id`
- Domain quota policy: `_domain_quota`
- Hard negative mining: `_pick_hard_negative`

Practical explanation from real implementation

The current strategy samples candidate negatives and scores lexical overlap with query tokens, then caches choices per query id for reproducibility.

This approach is intentionally runtime-efficient at `200000` pair scale and avoids expensive full-corpus BM25 rescoring for every row.

Validation pointers

- Pair totals: `artifacts/reports/data_preparation_report.json`
- Split correctness tests: `tests/test_pair_builder.py`

Beginner checkpoint

- Understand why split assignment is based on query id hash.
- Understand difference between random negative and hard negative.

Advanced checkpoint

- Evaluate how lexical-overlap hard negatives may bias against semantic-hard negatives.
- Explain when to replace this strategy with semantic mining or ANN pre-mining.

Tutorial: 04_contrastive_training_and_lora

04 - Contrastive Training and LoRA

What it is

Training updates embedding behavior with adapter-based fine-tuning (LoRA) using contrastive objectives.

Why it is used

LoRA reduces memory footprint and update cost while preserving a reusable base model. Contrastive objectives align query vectors with relevant documents and separate hard negatives.

How it appears in code

- PEFT-style trainer:
`src/domain_adapted_embedding_alignment/training/trainer.py`
- Unsloth path with SentenceTransformers trainer:
`src/domain_adapted_embedding_alignment/training/unsloth_trainer.py`
- Training entrypoint:
`src/domain_adapted_embedding_alignment/pipelines/train_model.py`
- Adapter-aware embedding load path:
`src/domain_adapted_embedding_alignment/retrieval/backend_factory.py`

Practical explanation from run outputs

From `artifacts/reports/training_report.json`:

- `total_steps`: 700
- `train_examples`: 159698
- `val_examples`: 2000
- `best_val_loss`: 0.06946182250976562
- `runtime.backend_used`: unsloth

From `artifacts/logs/01_train.log`:

- Unsloth initialized on RTX 4060 Laptop GPU
- `task_type` set to `FEATURE_EXTRACTION`

Beginner checkpoint

- Understand what an adapter is.
- Understand why train/validation are both required.

Advanced checkpoint

- Explain the tradeoff between eval frequency and runtime.
- Explain how `MultipleNegativesRankingLoss` interacts with hard negatives in batch construction.

Common pitfalls

- Missing or incompatible adapter path breaks tuned backend loading.
 - Confusing base-model performance with adapter performance if adapter load silently fails.
-

Tutorial:

05_backend_selection_peft_vs_unsloth

05 - Backend Selection: PEFT vs Unsloth

What it is

A backend resolution system that decides whether training runs with `peft` or `unsloth`.

Why it is used

Different hosts have different capabilities (CUDA, installed packages). Backend routing keeps behavior explicit and reproducible, including fallback semantics.

How it appears in code

- Resolver:
`src/domain_adapted_embedding_alignment/training/backend_selection.py`
- Runtime capability checks: `unsloth_runtime_status()`
- Strict mode behavior: `resolve_training_backend(..., strict_backend=True)`
- Trainer execution entry:
`src/domain_adapted_embedding_alignment/pipelines/train_model.py`

Practical explanation from run outputs

From `artifacts/reports/training_report.json` runtime metadata:

- `backend_requested: unsloth`
- `backend_used: unsloth`
- `fallback_reason: null`
- `unsloth_available: true`

This confirms no fallback occurred in the completed run.

Beginner checkpoint

- Understand `auto` vs explicit backend selection.
- Understand what strict backend mode is protecting.

Advanced checkpoint

- Design a deployment policy for mixed environments (GPU and CPU hosts).
- Define alerting rules when requested backend and used backend diverge.

Verification hooks

- Unit tests: `tests/test_backend_selection.py`
 - Runtime report: `artifacts/reports/training_report.json`
-

Tutorial:

06_retrieval_systems_bm25_dense_hybrid

06 - Retrieval Systems: BM25, Dense, Hybrid

What it is

A comparative retrieval layer with sparse lexical search, dense semantic search, and hybrid fusion.

Why it is used

No single retrieval method consistently dominates across all domains, latency budgets, and query distributions.

How it appears in code

- Sparse retriever: `src/domain_adapted_embedding_alignment/retrieval/bm25.py`
- Dense retriever: `src/domain_adapted_embedding_alignment/retrieval/dense.py`
- Hybrid fusion functions:
 - `src/domain_adapted_embedding_alignment/retrieval/hybrid.py`
 - `weighted_score_fusion`
 - `reciprocal_rank_fusion`

- Evaluation orchestration:
`src/domain_adapted_embedding_alignment/pipelines/evaluate_models.py`

Practical explanation from run outputs

From `artifacts/evaluation/retrieval_evaluation.json`:

- BM25: recall@10 0.9667 , mean latency 0.86ms
- Baseline dense: recall@10 1.0000 , MRR 0.8542 , mean latency 2902.25ms
- Tuned dense: recall@10 0.9750 , MRR 0.7556 , mean latency 47.33ms
- Hybrid tuned: recall@10 0.9833 , MRR 0.8107 , mean latency 56.04ms

Interpretation: baseline dense provides highest quality in this profile; tuned dense and hybrid tuned offer much lower latency.

Beginner checkpoint

- Understand lexical vs semantic retrieval behavior.
- Understand why latency must be reported with quality.

Advanced checkpoint

- Evaluate whether hybrid should be weighted differently by domain.
- Identify tail-latency outliers (`max_ms`) and potential causes.

Practical takeaway

Use `artifacts/evaluation/benchmark_table.csv` as deployment guidance, not leaderboard-only scoring.

Tutorial: 07_evaluation_metrics_and_judging

07 - Evaluation Metrics and LLM Judging

What it is

Evaluation layer that combines:

- retrieval quality metrics (`recall@k` , `precision@k` , `MRR` , `NDCG` , `MAP`)
- latency summaries (`mean` , `p50` , `p95` , `max`)
- RAG proxy metrics (`context_precision` , `context_recall`)
- optional LLM judge metrics

- embedding-space diagnostics (ranking accuracy and clustering summaries)

Why it is used

A retrieval system can look excellent on one metric and fail operationally or semantically. Multi-view evaluation prevents false confidence.

How it appears in code

- Evaluator core:
`src/domain_adapted_embedding_alignment/evaluation/evaluator.py`
- Metric implementation:
`src/domain_adapted_embedding_alignment/retrieval/metrics.py`
- Judge calls and fallback:
`src/domain_adapted_embedding_alignment/evaluation/llm_judge.py`
- Stage wrapper:
`src/domain_adapted_embedding_alignment/pipelines/evaluate_models.py`

Practical explanation from run outputs

From `artifacts/evaluation/retrieval_evaluation.json`:

- Judge metrics are present for baseline and tuned dense systems.
- Similarity ranking accuracy improved from `0.8933` to `0.9700`.

From `artifacts/logs/02_evaluate.log`:

- Repeated parse failures occurred for judge model `qwen3.5:4b`.
- Fallback behavior returned neutral scores so evaluation completed successfully.

Beginner checkpoint

- Understand what each retrieval metric measures.
- Understand why judge scores are auxiliary diagnostics.

Advanced checkpoint

- Identify when judge fallback should invalidate model comparisons.
- Define confidence intervals or bootstrap plans for retrieval metrics across domains.

Important interpretation note

Judge output is helpful but non-authoritative. Primary retrieval decisions should still rely on deterministic retrieval and latency metrics.

Tutorial: 08_embedding_space_visualization

08 - Embedding Space Visualization

What it is

Projection of high-dimensional embeddings into 2D using PCA, t-SNE, and UMAP for qualitative diagnostics.

Why it is used

Ranking metrics capture outcome quality but not geometric behavior. Projection plots can reveal clustering, overlap, or collapse patterns by domain.

How it appears in code

- Visualization utilities:
`src/domain_adapted_embedding_alignment/evaluation/visualization.py`
- Stage integration:
`src/domain_adapted_embedding_alignment/pipelines/run_end_to_end.py`
 - `plot_embedding_projections(..., prefix="baseline")`
 - `plot_embedding_projections(..., prefix="tuned")`

Practical explanation from run outputs

Generated outputs:

- `artifacts/figures/baseline_pca.png`
- `artifacts/figures/baseline_tsne.png`
- `artifacts/figures/baseline_umap.png`
- `artifacts/figures/tuned_pca.png`
- `artifacts/figures/tuned_tsne.png`
- `artifacts/figures/tuned_umap.png`

These figures are also validated by completion checks (`completion_checklist.json`).

Beginner checkpoint

- Understand why PCA is more stable and t-SNE/UMAP more local-structure focused.

Advanced checkpoint

- Compare projection narratives against quantitative metrics to avoid visual over-interpretation.

Tutorial: 09_rag_chroma_and_pinecone

09 - RAG with Chroma and Pinecone

What it is

Benchmark stage for retrieval-augmented generation contexts using:

- local Chroma collections
- optional Pinecone indexes

Why it is used

RAG systems need stable top-k retrieval quality and predictable latency under realistic index/query workloads.

How it appears in code

- Pipeline:
`src/domain_adapted_embedding_alignment/pipelines/run_rag_benchmarks.py`
- Chroma indexing/search:
`src/domain_adapted_embedding_alignment/rag/chroma_demo.py`
- Pinecone indexing/search:
`src/domain_adapted_embedding_alignment/rag/pinecone_demo.py`

Practical explanation from run outputs

From `artifacts/logs/03_rag.log`:

- Chroma baseline index built for 1500 docs
- Chroma tuned index built for 1500 docs

From `artifacts/evaluation/rag_benchmark.json`:

- Chroma baseline hit@5: 1.0
- Chroma tuned hit@5: 0.9416666666666667
- Chroma mean latency: baseline 159.95ms , tuned 45.06ms
- Pinecone: `executed=false (reason: disabled)`

Beginner checkpoint

- Understand difference between local vector store and managed vector DB.

Advanced checkpoint

- Decide when Pinecone should be mandatory instead of optional.
- Define SLOs using p95/p99 latency and hit-rate thresholds.

Operational note

Pinecone execution requires runtime enablement plus credentials.

Tutorial: 10_graphrag_local_and_global

10 - GraphRAG Local and Global Retrieval

What it is

A graph-augmented retrieval layer that adds two retrieval modes on top of dense seeds:

- local neighborhood expansion
- global community retrieval

Why it is used

Dense retrieval alone can miss structural relationships. Graph-based expansion can inject useful context and improve retrieval behavior in some settings.

How it appears in code

- Graph construction:
`src/domain_adapted_embedding_alignment/graphrag/graph_builder.py`
- Local graph retrieval: `local_graph_retrieval(...)` in
`src/domain_adapted_embedding_alignment/graphrag/retriever.py`
- Global community retrieval: `global_community_retrieval(...)` in
`src/domain_adapted_embedding_alignment/graphrag/retriever.py`
- Benchmark pipeline:
`src/domain_adapted_embedding_alignment/pipelines/run_graphrag_benchmarks.py`

Practical explanation from run outputs

From `artifacts/logs/04_graphrag.log`:

- Graph built with 9220 nodes, 187729 edges, 6 communities

From `artifacts/evaluation/graphrag_benchmark.json`:

- Baseline local/global hit@5: 0.9467 / 0.46
- Tuned local/global hit@5: 0.88 / 0.46
- Tuned dense latency mean: 54.48ms (vs baseline 157.65ms)

Beginner checkpoint

- Understand the difference between local expansion and global community retrieval.

Advanced checkpoint

- Assess whether global retrieval should include embedding features beyond term overlap.
- Evaluate graph construction sensitivity to term extraction heuristics.

Practical takeaway

In this run, local and dense latency improved for tuned path, but global hit@5 remained unchanged.

Tutorial: 11_inference_and_serving_patterns

11 - Inference and Serving Patterns

What it is

Query-time retrieval path that returns ranked document candidates with metadata and text previews.

Why it is used

This is the production-facing interface of the system: users submit queries, retrieval returns explainable ranked context.

How it appears in code

- Inference stage:
`src/domain_adapted_embedding_alignment/pipelines/run_inference.py`
- Query execution helper:
`src/domain_adapted_embedding_alignment/rag/inference.py`

- Tuned backend wiring: `build_tuned_backend(...)` in `src/domain_adapted_embedding_alignment/retrieval/backend_factory.py`

Practical explanation from run outputs

From `artifacts/logs/05_inference.log`:

- Example query count: 3
- Mean latency: 65.10ms
- Backend used: `HuggingFaceEmbeddingBackend`
- Output structure includes:
 - `rank`
 - `doc_id`
 - `score`
 - `domain`
 - `source`
 - `preview`

Beginner checkpoint

- Understand how top-k retrieval output differs from final answer generation.

Advanced checkpoint

- Define inference logging schema for offline error analysis and relevance drift detection.
- Design caching policy for repeated queries without degrading freshness.

Practical guidance

Persist query and top-k outputs in your serving layer so you can debug retrieval failures with evidence, not intuition.

Tutorial:

12_reports_validation_and_auditability

12 - Reports, Validation, and Auditability

What it is

Final-stage reporting and validation layer that confirms run completeness and captures tooling impact.

Why it is used

A run is not operationally complete until required artifacts exist, are non-empty, and runtime metadata is coherent.

How it appears in code

- Final report assembly:
`src/domain_adapted_embedding_alignment/pipelines/build_final_report.py`
- Completion checks: `build_completion_checklist(...)` in
`src/domain_adapted_embedding_alignment/pipelines/finalize_reports.py`
- Tool impact summary: `build_tool_impact_report(...)` in
`src/domain_adapted_embedding_alignment/pipelines/finalize_reports.py`

Practical explanation from run outputs

From `artifacts/reports/completion_checklist.json`:

- `all_required_artifacts_ok=true`
- `all_runtime_checks_ok=true`
- `project_completion_ok=true`

From `artifacts/reports/tool_impact_report.json`:

- `unsloth.used=true`
- `peft.used=false`
- `trl.used_in_runtime=false`

From `artifacts/reports/final_report.json`:

- Combined payload includes data prep, training, evaluation, RAG, GraphRAG, visualizations, inference, completion, and tool impact.

Audit checklist

1. Confirm all required artifacts exist and are non-empty.
2. Confirm runtime backend metadata exists in training report.
3. Confirm stage logs exist under `artifacts/logs/`.
4. Confirm tests passed (`artifacts/logs/09_pytest.log`).
5. Confirm notebook execution outputs exist (`notebooks/*.executed.ipynb`).

Practical outcome

This project run satisfies the completion gate and can be treated as an auditable end-to-end execution.